

Comparable<T>::compareTo(T other): returns <0 $\iff$ this is less than other.

```
8 */
9 class Queue<T> implements Comparable<Queue<T>> {
10     @Override
11     public int compareTo(Queue<T> other) {
12         return this.length() - other.length();
13     }
14     /** An array to store the items in the queue. */
```

To handle overriding to String():

```
6
7     @Override
8     public abstract String toString();
9
```

~probably bad practice.

ID allocation using glob's var:

```
4
5     private final int id;
6     private static int nextId = 0;
7
8     public Car() {
9         this.id = Car.nextId++;
10    }
11
```

Using static final variables:

```
2 class Junction {
3     public static final boolean CAR_CAN_CROSS = true;
4     public static final boolean PEDESTRIAN_CAN_CROSS = false;
5
```

```
10
11 sb.insert(2, "AAA"); // trAAAueabcdefg1
12 sb.replace(3, 4, "B"); // trABAueabcdefg1
13 sb.substring(3); // BAueabcdefg1
14 sb.lastIndexOf("e"); // 11
15 sb.indexOf("ABA"); // 2
16 sb.length(); // 15
17 }
```

Formatting:

Flag	General	Character	Integral	Floating Point	Date/Time	Description
'l'	y	y	y	y	y	The result will be left-justified.
'#'	y ¹	-	y ³	y	-	The result should use a conversion-dependent alternate form.
'+'	-	-	y ⁴	y	-	The result will always include a sign.
' '	-	-	y ⁴	y	-	The result will include a leading space for positive values.
'0'	-	-	y	y	-	The result will be zero-padded.
'.'	-	-	y ²	y ²	-	The result will include locale-specific grouping separators.
'('	-	-	y ⁴	y ²	-	The result will enclose negative numbers in parentheses.

String Builder

```
3 StringBuilder sb = new StringBuilder(); //
4 sb.append(true); // true
5 sb.append('a'); // truea
6 sb.append(new char[]{'a', 'b', 'c', 'd'}, 1, 2); // trueabc
7 sb.append("de"); // trueabcde
8 sb.append("00fg00", 2, 4); // trueabcfg
9 sb.append(1); // trueabcfg1
10
```

- The format specifiers for general, character, and numeric types have the following syntax:

`%[argument_index$][flags][width][.precision]conversion`

The optional *argument_index* is a decimal integer indicating the position of the argument in the argument list. The first argument is referenced by "1\$", the second by "2\$", etc.

The optional *flags* is a set of characters that modify the output format. The set of valid flags depends on the conversion.

The optional *width* is a positive decimal integer indicating the minimum number of characters to be written to the output.

The optional *precision* is a non-negative decimal integer usually used to restrict the number of characters. The specific behavior depends on the conversion.

The required *conversion* is a character indicating how the argument should be formatted. The set of valid conversions for a given argument depends on the argument's data type.

- The format specifiers for types which are used to represent dates and times have the following syntax:

`%[argument_index$][flags][width]conversion`

The optional *argument_index*, *flags* and *width* are defined as above.

The required *conversion* is a two character sequence. The first character is 't' or 'T'. The second character indicates the format to be used. These characters are similar to but not completely identical to those defined by GNU date and POSIX strftime(3c).

- The format specifiers which do not correspond to arguments have the following syntax:

`%[flags][width]conversion`

The optional *flags* and *width* is defined as above.

The required *conversion* is a character indicating content to be inserted in the output.

```
'R' Time formatted for the 24-hour clock as "%tH:%tM"
'T' Time formatted for the 24-hour clock as "%tH:%tM:%tS".
'r' Time formatted for the 12-hour clock as "%tI:%tM:%tS %tP". The location of the morning or afternoon marker ('%tP') may be locale-dependent.
'D' Date formatted as "%tm/%td/%ty".
'F' ISO 8601 complete date formatted as "%tY-%tm-%td".
'c' Date and time formatted as "%ta %tb %td %tT %tZ %tY", e.g. "Sun Jul 20 16:17:00 EDT 1969".
```

Conversion	Argument Category	Description
'b', 'B'	general	If the argument <i>arg</i> is null, then the result is "false". If <i>arg</i> is a boolean or Boolean, the result is "true".
'h', 'H'	general	If the argument <i>arg</i> is null, then the result is "null". Otherwise, the result is obtained by formatting <i>arg</i> as a hexadecimal integer.
's', 'S'	general	If the argument <i>arg</i> is null, then the result is "null". If <i>arg</i> implements Formattable, the result is obtained by formatting <i>arg</i> as a string.
'c', 'C'	character	The result is a Unicode character.
'd'	integral	The result is formatted as a decimal integer.
'o'	integral	The result is formatted as an octal integer.
'x', 'X'	integral	The result is formatted as a hexadecimal integer.
'e', 'E'	floating point	The result is formatted as a decimal number in computerized scientific notation.
'f'	floating point	The result is formatted as a decimal number.
'g', 'G'	floating point	The result is formatted using computerized scientific notation or decimal format, depending on the precision and the value of the argument.
'a', 'A'	floating point	The result is formatted as a hexadecimal floating-point number with a significand and an exponent, the latter's being in the <i>floating point</i> argument category.
't', 'T'	date/time	Prefix for date and time conversion characters. See Date/Time Conversions.
'%'	percent	The result is a literal '%' ('%0025').
'n'	line separator	The result is the platform-specific line separator.

Making Generic Access::

```
this.item = 0;
@SuppressWarnings("unchecked")
T[] temp = (T[]) new Object[size];
this.items = temp;
```

- ① WRITE COMMENT
- ② TYPE SAFE
- ③ MOST LIMITED SCOPE.

EXCEPTIONS.

Try to keep repeated strings within the constructor itself

```
// initialised to be a queue of passengers next.  
@SuppressWarnings("unchecked")  
Queue<Passenger>[] temp = (Queue<Passenger>[]) new Queue<?>[numStops];  
this.passengers = temp;  
for (int i = 0; i < numStops; i++) {  
    this.passengers[i] = new Queue<Passenger>(capacity);  
}
```

```
Array(int size) {  
    // The only way we can put an object into the array is through  
    // the method set() and we only put an object of type T inside.  
    // So it is safe to cast 'Object[]' to 'T[]'.  
    @SuppressWarnings("unchecked")  
    T[] a = (T[]) new Object[size];  
}
```

DO BEFORE PE:

```
87 let g:airline#extensions#tabline#enabled = 1  
NORMAL vimrc
```

```
38 set termguicolors  
39 " Use gruvbox as default color scheme  
40 color gruvbox  
41 set background=dark  
42 " To get around problem with seemingly r
```

```
dconf read /org/gnome/desktop/input-sources/xkb-options  
dconf write /org/gnome/desktop/input-sources/xkb-options "['caps:escape']"
```

```
49  
50 " some advanced options requested by students. Use them only if  
51 " you know what you are doing (I don't know what they do!)  
52 set wildmenu  
53 set showcmd  
54 " set hlsearch
```

```
50 set expandtab  
31 " Display lines longer than the current window on the next line(s)  
32 " set wrap  
33 " Prevent breaking a word into multiple lines when wrapping  
34 " set linebreak  
35 " Display line and column number on the lower-right corner.
```